AI POWER OPTIMIZER

Intelligent Household Backup Power Management using Simulated Annealing

- Department: **Computer Science and Engineering**
- Subject: **Foundations of Artificial Intelligence**
- Domain: **Artificial Intelligence (AI) – Local Search and Optimization**

PROJECT TEAM MEMBERS

Member 1
Vishnu Priyan M
Roll No: 240701604

Member 2
Thamizharasan M
Roll No: 240701560

Dept. of Computer Science & Engineering

Introduction & Problem Statement

The Household Power Outage Challenge

• Limited Backup Capacity: Inverter power (watts) and battery energy storage (watt-hours) are strictly bounded during grid failure.

• Simultaneous Load Conflict: Backup systems cannot operate all household appliances at the same time without tripping.

• Priority Imbalance: Critical loads (refrigerator, medical devices, Wi-Fi) must take precedence over heavy comfort loads (AC, heater).

• Energy Depletion Risk: Uncontrolled running depletes the battery early in the outage, leaving the household powerless later.

• Inefficiency of Manual Control: Manual switching fails to anticipate future energy needs, causing frequent overloads and wasted capacity.

CORE PROJECT OBJECTIVE

"To optimize household appliance operating schedules during power outages using Simulated Annealing while considering appliance priorities, inverter capacity, and available battery energy."

System Operational Model

Inverter Limit (W): Guarantees per-slot instantaneous power <= maximum rated inverter wattage.

Battery Storage (Wh): Total energy consumed over outage <= available stored battery reserve.

Essential Runtimes: Guarantees minimum operating duration for essential loads before outage ends.

Objectives & Proposed Solution

Key Project Objectives

1. Prioritize Essential Appliances
   Ensure critical devices (refrigerator, Wi-Fi) receive uninterrupted or guaranteed runtimes.

2. Determine ON/OFF Schedules
   Compute precise time-slot schedules for each appliance across the outage period.

3. Allocate Operating Duration
   Fulfill required hours for appliances without exceeding user-specified limits.

4. Respect Inverter Power Limits
   Never exceed peak inverter wattage (e.g., 1000 W) in any discrete time slot.

5. Conserve Battery Energy
   Prevent premature battery exhaustion by keeping total Wh usage within storage capacity.

6. User Review & Visualization
   Present intuitive Gantt charts, energy graphs, and natural-language explanations.

Proposed Solution Architecture

Stage 1: User Input & Specification

Household enters battery capacity (Wh), inverter limit (W), outage hours, and appliance list with power ratings, priority levels, and duration requirements.

Stage 2: AI Optimization (Simulated Annealing)

System validates constraints, generates an initial feasible schedule, and iteratively explores neighbor configurations using temperature-guided local search.

Stage 3: Recommended Schedule & Telemetry

Outputs an optimal timetable, peak load metrics, battery depletion profiles, comparison vs. naive baseline, and explanations for each decision.

Foundations of AI: Syllabus Mapping

Unit I: Introduction to Artificial Intelligence

- Intelligent Agent:
  The Power Optimizer senses battery state, inverter specs, and appliances, taking actions via scheduling.
- Agent and Environment:
  Environment = battery capacity, inverter limits, appliance profiles. Agent = SA optimizer.
- Rational Behaviour:
  Maximizes utility (fitness function) while adhering to power and energy constraints.
- Problem-Solving Agent:
  Formulates scheduling as a constrained optimization search problem to reach an optimal goal state.

Unit II: Problem Solving (Core Project Focus)

- Local Search Strategy: Maintains a single current candidate schedule and iteratively explores neighbors.
- State Space Formulation: Combinatorial state space of 2^(appliances × time slots) possible binary combinations.
- Neighbor Generation: Six stochastic mutation operators modify slots, durations, or appliance timings.
- Objective Function: Multi-objective fitness balancing priority weights, runtimes, and hard penalty terms.
- Simulated Annealing: Core search algorithm with probabilistic acceptance escaping local optima.

Note on Syllabus Scope: Unit II is the core foundation. Game playing (Unit III) and logical reasoning/inference engines (Unit IV) are not required for the optimization pipeline.

System Architecture

1. User & Interface

• User:
Interacts via browser dashboard.

• Input Interface:
Interactive web UI built with HTML5, CSS3, and JavaScript.

• Appliance & Battery Data:
Structured JSON payload capturing wattage, duration, priority, and battery specs.

2. Validation & Pre-check

• Data Validation:
Pydantic models validate data types, range bounds, and non-empty appliance lists.

• Constraint Checker:
constraints.py enforces hard inverter peak limits and battery energy bounds.

• FastAPI Backend:
Asynchronous REST endpoints orchestrate the optimization pipeline.

3. SA Engine (simulated_annealing.py)

• Initial State Generator:
scheduler.py generates random initial feasible schedule.

• Neighbor Generator:
neighbor.py executes stochastic mutations on schedule slots.

• Fitness Evaluator:
fitness.py computes multi-factor score and constraint penalties.

• Best Solution Tracker:
Monitors best-so-far schedule across all iterations.

4. Output & Dashboard

• Optimized Power Schedule:
Final binary ON/OFF matrix formatted with time intervals.

• Interactive Dashboard:
Chart.js v4 rendering power demand curves and battery discharge profiles.

• Gantt Chart:
Visual timeline mapping appliance active intervals.

• Comparative Analysis:
Side-by-side performance comparison against naive baseline.

Simulated Annealing Algorithm

Algorithm Definition & Process

"Simulated Annealing is a local search optimization algorithm inspired by the controlled cooling of heated materials."

1. Generate initial schedule (randomized priority heuristic)
2. Set initial temperature T = T0 (1000)
3. Calculate fitness of current schedule
4. Generate neighbor schedule via mutation operator
5. Calculate neighbor schedule fitness
6. If neighbor is better (ΔE > 0), ALWAYS accept it
7. If worse (ΔE <= 0), accept with probability P = exp(ΔE / T)
8. Update best-ever solution if current fitness exceeds it
9. Cool temperature: T = T × α (geometric cooling)
10. Repeat until T < T_min or max iterations reached; return best

METROPOLIS ACCEPTANCE CRITERION

$$P = \exp(\Delta E / T)$$

- ΔE = New Fitness − Current Fitness (negative value)
- T = Current Temperature
- High T (Early): P ≈ 1 → Explores broadly, escapes local optima
- Low T (Late): P ≈ 0 → Exploits best discovered region

Documented Hyperparameters

- Initial Temperature (T0): 1000 (Controls initial exploration intensity)
- Minimum Temperature (T_min): 0.1 (Stopping convergence threshold)
- Cooling Rate (α): 0.95 (Geometric cooling decay factor)
- Maximum Iterations: 10,000 (Upper computational iteration limit)

State Representation & Neighbor Generation

2D Binary Matrix State Representation

- State definition: schedule[appliance][slot] ∈ {0, 1}
- 1 = Appliance is ON during time slot; 0 = Appliance is OFF
- Rows = appliances; Columns = discrete time slots (e.g. 30 min each)

Appliance	T1	T2	T3	T4	T5	T6
Refrigerator (150W)	1	1	1	1	1	1
Wi-Fi Router (10W)	1	1	1	1	1	1
Water Pump (750W)	0	1	1	0	0	0

** Illustrative 3-appliance × 6-slot schedule state. Search space size for 6 appliances & 8 slots = 2^48 ≈ 2.81 × 10^14 combinations.*

Neighbor Generation Operators

- 1. Toggle Slot:
  Flip a single cell (0 -> 1 or 1 -> 0) for a randomly selected appliance.
- 2. Move Operation:
  Shift an operating interval to another start slot, keeping total runtime constant.
- 3. Extend Duration:
  Turn ON an adjacent slot to satisfy higher runtime requirement.
- 4. Reduce Duration:
  Turn OFF an active slot to alleviate battery or inverter overload.
- 5. Swap Schedules:
  Exchange active operating slots between two compatible appliances.

Fitness Function & Constraints

Fitness Function Formulation

Fitness = Σ(Rewards) − Σ(Penalties)

- Priority Reward: Higher score for powering essential & high-priority appliances.
- Duration Satisfaction: Reward proportional to meeting requested operating hours.
- Time Preference Reward: Bonus for running within preferred time-of-day windows.
- Power Violation Penalty: -10,000 heavy penalty per time-slot exceeding inverter limit.
- Energy Violation Penalty: -5,000 penalty for exceeding total available battery energy.
- Switching Penalty: -2 penalty per ON/OFF transition to minimize frequent toggling.

The Five System Constraints

C1: Inverter Power Limit
Per-slot total power consumption <= max inverter rated wattage (1000 W).

C2: Battery Energy Reserve
Cumulative energy consumed <= total available battery capacity (Wh).

C3: Essential Appliance Runtime
Essential appliances must satisfy minimum designated operating hours.

C4: Preferred Time Window
Appliances with preferred hours operate only inside designated intervals.

C5: Maximum Duration Bound
Operating duration for any appliance must not exceed max allowed hours.

Technology Stack & Implementation Modules

Frontend Layer

- HTML5: Clean semantic structure with glassmorphic cards.
- Vanilla CSS: Dark glassmorphic styling, responsive layout.
- Vanilla JavaScript: Asynchronous API calls, DOM rendering.
- Chart.js v4: Real-time Gantt schedule & telemetry charts.

Backend Layer

- Python 3.8+: Core execution language and data models.
- FastAPI: High-performance ASGI REST API framework.
- Pydantic: Schema validation & serialization for request models.
- Uvicorn: Lightning-fast ASGI production web server.

AI Algorithm Engine (simulated_annealing.py)

- 100% Hand-implemented Simulated Annealing algorithm.
- Custom fitness evaluator (fitness.py) with soft rewards & penalties.
- Custom hard constraint checker (constraints.py).
- Custom stochastic neighbor generator (neighbor.py).

Data Management & Key Distinction

- JSON Storage: Lightweight configuration via default_appliances.json.
- Zero Database Overhead: Pure in-memory computation.
- NO External Solvers: Strictly avoids OR-Tools, PuLP, SciPy optimize.
- Academic Authenticity: Clean, transparent code suitable for viva demo.

Demonstration Scenario

Household Backup Configuration

- Inverter Power Limit: 1000 Watts maximum continuous load
- Battery Storage Capacity: 3000 Watt-hours (Wh) total
- Initial Battery Charge: 70% SoC at onset of outage
- Available Battery Energy: 2100 Watt-hours (Wh) available
- Outage Duration: 4.0 Hours (8 slots @ 30 min each)

Peak Slot Calculation (Illustrative):
750W (Pump) + 150W (Fridge) + 75W (Fan) + 10W (Wi-Fi) = 985 Watts
✓ 985 W ≤ 1000 W inverter ceiling. Safely operable without tripping the inverter.

Household Appliances Profile

Appliance	Power	Priority	Essential?
Refrigerator	150 W	VERY HIGH	Yes (Essential)
Wi-Fi Router	10 W	HIGH	Yes (Essential)
Ceiling Fan	75 W	HIGH	No
Television	120 W	MEDIUM	No
Water Pump	750 W	MEDIUM	No
Air Conditioner	1200 W	LOW	No

Key Observations:
- Air Conditioner (1200W) > 1000W Inverter Limit -> Automatically excluded.
- Water Pump (750W) can only run when other heavy loads are turned off.

Results, Evaluation & Future Scope

Documented System Outputs

- Optimized Schedule Matrix:
  Gantt schedule of active appliance intervals.
- Energy & Power Telemetry:
  Total energy consumed (Wh), remaining battery (Wh & %), peak load (W).
- Satisfaction Metrics:
  Priority satisfaction index (%) and essential load satisfaction (%).
- Natural Language Explanations:
  Human-readable justifications for every appliance decision.
- Validation & Benchmark:
  Automated feasibility tests and comparison against naive scheduler.

Planned Future Enhancements

- Real-Time IoT Monitoring:
  Direct integration with smart battery BMS and IoT energy meters.
- Hybrid Power Sources:
  Dynamic dispatch across solar PV, wind microgeneration, and grid power.
- Weather-Based Solar Forecasting:
  Predict solar insolation curves to proactively adjust daytime backup.
- Mobile Client Application:
  Flutter / React Native mobile interface with push alert notifications.
- Multi-Day Outage Planning:
  Dynamic horizon planning for extended multi-day disaster scenarios.
- Appliance Usage Learning:
  Machine learning of household usage routines to automate preference settings.

Conclusion & Project Summary

Key Takeaways

• Practical Problem Solving: Solves real-world household backup energy bottlenecks during unexpected grid failures.

• Principled Local Search: Demonstrates Simulated Annealing from Foundations of AI Unit II, effectively navigating combinatorial state spaces.

• Balanced Optimization: Multi-factor objective function respects critical priorities while strictly preventing inverter trip and battery exhaustion.

• Rigorous Constraint Validation: Hard penalty formulation guarantees physically viable schedules (under 1000W peak load).

• Transparent & Extensible: Zero black-box solvers; hand-crafted algorithms with natural-language explanation capability.

"AI-driven scheduling can help households make better use of limited backup power during outages."

THANK YOU!

Questions & Discussion